

Compare-And-Set, ABA and Advanced Lock-Free Concurrency

This chapter focuses on the mechanisms that make lock-free updates possible in Java. It begins with Compare-And-Set, then moves into retry loops, contention, atomic arrays, memory visibility, the ABA problem and high-contention counters.

The central idea is simple:

Read the current state, calculate the desired state, and update it only if no other thread changed it in the meantime.

These techniques are powerful, but they should be used only when the shared state can be expressed as a small atomic transition. More complex invariants may still be clearer and safer with locks or concurrent collections.

1. Compare-And-Set (CAS)

1.1 What Is CAS?

CAS stands for **Compare-And-Set** or **Compare-And-Swap**.

The Java method is named:

```
compareAndSet(expectedValue, newValue)
```

Its meaning is:

```
If current value equals expected value:
    replace it with new value
    return true
```

```
Otherwise:
```

```
leave it unchanged  
return false
```

CAS combines comparison and update into one atomic operation.

1.2 Simple CAS Example

```
AtomicInteger number =  
    new AtomicInteger(10);  
  
boolean changed =  
    number.compareAndSet(10, 20);  
  
System.out.println(changed);  
System.out.println(number.get());
```

Output:

```
true  
20
```

Failure case:

```
AtomicInteger number =  
    new AtomicInteger(30);  
  
boolean changed =  
    number.compareAndSet(10, 20);
```

Output state:

```
changed = false  
number = 30
```

1.3 CAS as Optimistic Concurrency

CAS uses an optimistic strategy.

```
I read state X
    ↓
I calculate a new state Y
    ↓
I assume nobody changed X
    ↓
I attempt CAS(X, Y)
```

If the assumption is correct, the update succeeds.

If another thread changed the value first, the update fails and the thread can retry.

This differs from pessimistic locking:

```
Lock first
    ↓
Prevent others from entering
    ↓
Read and update
    ↓
Unlock
```

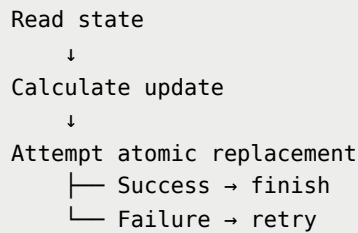
1.4 Lock vs CAS

Lock-Based Update

```
Acquire lock
    ↓
Read state
    ↓
Modify state
    ↓
Release lock
```

Other threads attempting to enter may block.

CAS-Based Update



Several threads may attempt the operation, but only an update based on the current expected value succeeds.

1.5 How CAS Is Implemented

At the Java level, atomic classes expose operations such as:

```
compareAndSet()  
getAndAdd()  
getAndSet()
```

The JVM usually maps these operations to platform-specific atomic instructions or intrinsics.

On modern x86 processors, a CAS operation may use an instruction such as locked `CMPXCHG`.

The important points are:

- comparison and update are performed atomically
- the operation participates in the processor's cache-coherence protocol
- competing cores cannot both successfully update the same memory location from the same expected state
- the JVM provides the Java Memory Model guarantees required by the atomic API

It is usually inaccurate to imagine that every CAS operation locks the entire system bus. Modern processors commonly coordinate atomic updates at the cache-line and coherence-protocol level.

Application code should rely on Java's documented atomicity and memory-ordering guarantees rather than on one specific CPU implementation.

1.6 CAS Retry Loop

CAS can fail when another thread changes the value after it was read.

A retry loop handles this situation.

```
while (true) {  
    int current = value.get();  
    int updated = calculate(current);  
  
    if (value.compareAndSet(  
        current,  
        updated  
    )) {  
        break;  
    }  
}
```

On failure:

```
The previously read state is stale  
    ↓  
Read the latest state  
    ↓  
Recalculate the update  
    ↓  
Try again
```

This pattern is the foundation of many lock-free algorithms.

1.7 Like Counter Using a CAS Loop

An `AtomicInteger` would be the simplest implementation for an integer counter, but a manual CAS loop helps demonstrate the algorithm.

```
import java.util.concurrent.atomic.AtomicInteger;

class LikeCounter {

    private final AtomicInteger totalLikes =
        new AtomicInteger(0);

    public void like() {

        while (true) {
            int current = totalLikes.get();
            int updated = current + 1;

            if (totalLikes.compareAndSet(
                current,
                updated
            )) {
                return;
            }
        }

        public int getTotalLikes() {
            return totalLikes.get();
        }
    }
}
```

Test:

```
public class LikeCounterDemo {

    public static void main(String[] args)
        throws InterruptedException {

        LikeCounter counter = new LikeCounter();
    }
}
```

```
Thread[] threads = new Thread[7];

for (int i = 0; i < threads.length; i++) {
    threads[i] =
        new Thread(counter::like);

    threads[i].start();
}

for (Thread thread : threads) {
    thread.join();
}

System.out.println(
    "Total likes: " +
    counter.getTotalLikes()
);
}
```

Output:

```
Total likes: 7
```

For this exact operation, prefer:

```
totalLikes.incrementAndGet();
```

It is clearer and allows the JVM to use the most efficient supported atomic operation.

1.8 Do Not Use `Thread.sleep()` for Coordination

This is unreliable:

```
firstThread.start();
secondThread.start();

Thread.sleep(3000);

System.out.println(counter.get());
```

Three seconds may be:

- more than necessary
- less than necessary
- affected by machine load
- affected by scheduling

Use `join()` when waiting for threads:

```
firstThread.join();
secondThread.join();
```

For executor tasks, use:

- `Future.get()`
- `CompletableFuture.join()`
- `ExecutorService.awaitTermination()`
- coordination utilities such as `CountDownLatch`

1.9 CAS Contention

Under low contention, a CAS update can complete quickly.

Under high contention:

```
Many threads read the same old state
↓
One update succeeds
↓
```

Many updates fail
↓
Threads retry
↓
CPU time is spent recalculating and retrying

CAS avoids lock ownership, but it does not eliminate contention.

Possible effects include:

- wasted CPU cycles
- cache-line bouncing between cores
- increased latency
- starvation of unlucky threads
- reduced throughput under extreme contention

Lock-free is not automatically faster than a well-designed lock.

1.10 Starvation

A lock-free algorithm guarantees system-wide progress, not necessarily progress for every thread.

Thread A succeeds repeatedly
Thread B repeatedly loses CAS

Thread B may retry many times.

This is one reason lock-free does not mean wait-free.

When fairness is a requirement, an explicit fair lock or another coordination mechanism may be more suitable.

1.11 Backoff in Retry Loops

A tight retry loop can consume significant CPU during contention.

```
while (!reference.compareAndSet(  
    expected,
```

```
        updated
    )) {
        // retry
    }
```

For custom low-level algorithms, techniques may include:

- rereading the latest value
- `Thread.onSpinWait()`
- bounded spinning
- yielding
- exponential backoff
- eventually falling back to a lock

Example:

```
while (true) {
    State current = state.get();
    State updated = calculate(current);

    if (state.compareAndSet(
        current,
        updated
    )) {
        return;
    }

    Thread.onSpinWait();
}
```

Do not add spinning mechanically. It must be justified by measurement and expected contention duration.

2. Atomic Arrays

2.1 Why Atomic Arrays Exist

A normal array does not make updates to its elements safely composable across threads.

```
int[] counters = new int[10];

counters[0]++;
```

The increment is still a non-atomic read-modify-write operation.

Java provides:

```
AtomicIntegerArray
AtomicLongArray
AtomicReferenceArray<E>
```

Each element can be accessed and updated atomically.

2.2 `AtomicIntegerArray`

```
import java.util.concurrent.atomic.AtomicIntegerArray;

AtomicIntegerArray counters =
    new AtomicIntegerArray(3);

counters.incrementAndGet(0);
counters.addAndGet(1, 5);

System.out.println(counters.get(0));
System.out.println(counters.get(1));
```

Each index behaves like an independent atomic value.

2.3 `AtomicReferenceArray`

Coder Army

```
import java.util.concurrent.atomic.AtomicReferenceArray;

AtomicReferenceArray<String> values =
    new AtomicReferenceArray<>(3);

values.set(0, "A");
values.set(1, "B");
values.set(2, "C");

System.out.println(values.get(1));
```

Conditional update:

```
boolean changed =
    values.compareAndSet(
        1,
        "B",
        "X"
    );
```

This means:

If index 1 currently contains the expected reference,
replace it with X.

As with `AtomicReference`, expected object values are matched by reference identity.

2.4 Theatre Seats With `AtomicReferenceArray`

```
import java.util.concurrent.atomic.AtomicReferenceArray;

class MovieTheatre {

    private final AtomicReferenceArray<String> seats;
```

```
MovieTheatre(int totalSeats) {
    seats =
        new AtomicReferenceArray<>(
            totalSeats
        );
}

public boolean bookSeat(
    int seatNumber,
    String userName
) {
    validateSeatNumber(seatNumber);

    return seats.compareAndSet(
        seatNumber,
        null,
        userName
    );
}

public String getBookedBy(int seatNumber) {
    validateSeatNumber(seatNumber);
    return seats.get(seatNumber);
}

private void validateSeatNumber(
    int seatNumber
) {
    if (
        seatNumber < 0 ||
        seatNumber >= seats.length()
    ) {
        throw new IllegalArgumentException(
            "Invalid seat number: " +
            seatNumber
        );
    }
}
```

```

    }
}
};

```

Usage:

```

MovieTheatre theatre =
    new MovieTheatre(100);

boolean booked =
    theatre.bookSeat(
        25,
        "Harshita"
    );

if (booked) {
    System.out.println("Seat confirmed");
} else {
    System.out.println("Seat already booked");
}

```

Each seat can be updated independently.

Booking several seats as one indivisible transaction would require a broader coordination strategy.

3. **volatile** vs Atomic Variables

3.1 What **volatile** Provides

A volatile field provides visibility and ordering guarantees.

```

private volatile boolean running = true;

```

When one thread writes:

```
running = false;
```

other threads reading the field are guaranteed to observe the volatile update according to the Java Memory Model.

A volatile read and write of the field itself is atomic for supported field types.

3.2 What **volatile** Does Not Provide

volatile does not make a compound read-modify-write operation atomic.

```
private volatile int count = 0;

public void increment() {
    count++;
}
```

count++ is still conceptually:

```
read
modify
write
```

Two threads can still lose an update.

3.3 Volatile vs Atomic

Requirement	volatile	Atomic class
Latest value is visible across threads	Yes	Yes
Atomic plain read/write	Yes	Yes
Atomic increment	No	Yes
Atomic compare-and-set	No	Yes
Atomic functional update	No	Yes
Supports compound state transition	No	For one atomic value

Use `volatile` for a visibility-based state flag when no compound atomic transition is required.

Use an atomic class for operations such as:

```
increment
decrement
get-and-set
compare-and-set
update based on current value
```

3.4 Memory Visibility of Atomic Classes

Atomic variables provide more than indivisible updates.

Their reads and writes have volatile-like memory effects.

A successful update can safely publish state that was established before the update, provided the design follows the Java Memory Model correctly.

Example:

```
class ConfigurationHolder {

    private String configuration;

    private final AtomicBoolean ready =
        new AtomicBoolean(false);

    public void initialize() {
        configuration = "production";
        ready.set(true);
    }

    public String getConfiguration() {

        if (!ready.get()) {
            throw new IllegalStateException(
                "Not ready"
            );
        }

        return configuration;
    }
}
```

```

        );
    }

    return configuration;
}
}

```

The atomic flag supplies visibility, but this design still needs careful lifecycle control to ensure initialization happens only once.

4. The ABA Problem

4.1 What CAS Actually Checks

CAS asks:

Is the current value equal to my expected value now?

It does not automatically record:

- whether the value changed earlier
- how many times it changed
- which intermediate values existed

This can create the ABA problem.

4.2 ABA Sequence

Initial state:

A

Thread A reads:

A

Before Thread A updates it, Thread B changes:

```
A → B → A
```

Thread A then performs:

```
CAS(A → X)
```

The operation succeeds because the current value is again **A**.

Thread A cannot tell that the state changed temporarily.

4.3 Why ABA Can Matter

In a simple counter, changing from **10** to **11** and back to **10** may not matter.

In a lock-free data structure, the intermediate changes may represent important history.

For example:

```
Node A is at the top of a stack
    ↓
Node A is removed
    ↓
Memory or node is reused
    ↓
A reference resembling the old state appears again
```

A thread may incorrectly assume that the structure is unchanged because the reference looks identical.

4.4 Versioning Solution

Instead of storing only a value:

```
A
```

store the value with a version:

```
(A, 1)
```

Every change increments the version:

```
(A, 1) → (B, 2) → (A, 3)
```

Thread A expects:

```
(A, 1)
```

The current state is:

```
(A, 3)
```

The CAS operation fails even though the visible value is again `A`.

Final Takeaway

CAS is the foundation of many lock-free operations in Java:

```
Read current state
    ↓
Calculate desired state
    ↓
Compare current state with expected state
    ↓
Update only when they still match
```

This approach works best for small, clearly defined state transitions. It becomes less attractive when several mutable objects, external side effects or complex business rules must be coordinated together.

The correct question is not:

| Can this be written without a lock?

The better question is:

| Which concurrency mechanism makes the invariant correct, understandable and efficient?

Coder Army